

RECOVERY

BCSE302L – Database Systems
Part 2

CONTENTS

- Recovery concepts
- No-undo/redo Recovery Based on Deferred update
- Recovery technique Based on Immediate update
- Shadow paging

INTRODUCTION

- Database systems, like any other computer system, are subject to failures.
- Despite this, any organization that depends upon a database must have that database available when it is required.
- Therefore, any DBMS intended for a serious business or organizational user **must have adequate facilities for fast recovery after failure.**
- In particular, whenever a transaction is submitted to a DBMS for execution, the **system must ensure that either all the operations in the transaction are completed successfully and their effect is recorded permanently in the database,** or the transaction has no effect whatsoever on the database or on any other transactions.

INTRODUCTION

- **Recovery mechanisms** include automatic protection mechanisms built into the database software itself, and non-automatic actions available to people responsible for the running of the database system, **both for the backing up of data and recovery for a variety of failure situations.**
- **Recovery techniques** are **intertwined with the concurrency control mechanisms**: certain recovery techniques are best **used with specific concurrency control methods.**
- Assume for the most part that we are dealing with large multi-user databases.
- Small systems typically provide little or no support for recovery; in these systems, recovery is regarded as a user problem.

TYPICAL RECOVERY PROBLEM

- Data updates made by a DBMS are not automatically written to disk at each synchronization point.
- Therefore there may be some delay between the commit and the actual disk writing (i.e. regarding the changes as permanent and worthy of being made to disk).
- If there is a system failure during this delay, the system must still be able to ensure that these updates reach the disk copy of the database.

TYPICAL RECOVERY PROBLEM

- Conversely, data changes that may ultimately prove to be incorrect, made for example by a transaction that is later rolled back, can sometimes be written to disk.
- Ensuring that only the results of complete transactions are committed to disk is an important task, which if inadequately controlled by the DBMS may lead to problems, such as the generation of an inconsistent database.

EXAMPLE

- Suppose we want to enter a transaction into a customer order. The following actions must be taken:

START

- 1. Change the customer record with the new order data*
- 2. Change the salesperson record with the new order data*
- 3. Insert a new order record into the database*

- *STOP*

EXAMPLE

- Initial values are

CUSTOMER:

C-No	Order-No	Description	Cost
123	1,000	400 Tennis balls	£2,400

SALESPERSON:

Name	Total-Sales
Jones	£3,200

ORDERS:

Order-No
1000
2000
3000
4000
5000

EXAMPLE

- If only operations 1 and 2 are successfully performed, this results in the values shown in the figure
- Initial Value

Update Value

CUSTOMER:

C-No	Order-No	Description	Cost
123	1,000	400 Tennis balls	£2,400

SALESPERSON:

Name	Total-Sales
Jones	£3,200

ORDERS:

Order-No
1000
2000
3000
4000
5000

CUSTOMER:

C-No	Order-No	Description	Cost
123	1,000	400 Tennis balls	£2,400
123	8,000	250 Cricket balls	£6,500

SALESPERSON:

Name	Total-Sales
Jones	£9,700

ORDERS:

Order-No
1000
2000
3000
4000
5000

EXAMPLE

- This database state is clearly unacceptable as it does not accurately reflect reality; for example, a customer may receive an invoice for items never sent, or a salesman may make commission on items never received.
- It is better to treat the whole procedure (i.e. from START to STOP) as a complete transaction and not commit any changes to the database until STOP has been successfully reached.



LOG BASED RECOVERY PROTOCOL

TRANSACTION LOGGING

- System log
- Committing transactions and force-writing
- Checkpoints
- Undoing
- Redoing

TRANSACTION LOGGING

- System log
- Recovery manager overcomes many failures by different techniques using a special file known as a **system log or log or journal or audit trail**
- It contains information about start and end of each transaction and any updates which occur in the transaction.
- The **log is kept on disk** (apart from the most recent log block that is in the process of being generated, are stored in main memory).
- Log is not affected by failures, except for disk failure or catastrophic failure.
- Log is **periodically backed up** to storage.

TRANSACTION LOGGING

- System log
- Types of entries that are written to the log for transaction.
- In these entries, T refers to a unique transaction identifier that is generated automatically by the system and used to uniquely label each transaction.
- **start_transaction(T)**: This log entry records that transaction T starts the execution.
- **read_item(T, X)**: This log entry records that transaction T reads the value of database item X.

TRANSACTION LOGGING

- System log
- Types of entries that are written to the log for transaction.
- **write_item(T, X, old_value, new_value)**: This log entry records that transaction T changes the value of the database item X from old_value to new_value. The old value is sometimes known as a before image of X (BFIM), and the new value is known as an after image of X (AFIM).
- **commit(T)**: This log entry records that transaction T has completed all accesses to the database successfully and its effect can be committed (recorded permanently) to the database.
- **abort(T)**: This records that transaction T has been aborted.
- **checkpoint**: This is an additional entry to the log

TRANSACTION LOGGING

- **System log**
- Log contains a record of every write operation that changes the value of some database item, it is possible to undo the effect of these write operations of a transaction T by tracing backward through the log and resetting all items changed by a write operation of T to their **old_values**.
- Redo can be done with the effect of the write operations of a transaction T by tracing forward through the log and setting all items changed by a write operation of T to their **new_values**.
- Redoing the operations of a transaction may be required if all its updates are recorded in the log but a failure occurs before we can be sure that all the **new_values** have been written permanently in the actual database.

TRANSACTION LOGGING

- Committing Transactions
- A transaction T reaches its commit point when all its operations that access the database have been executed successfully; that is, the transaction has reached the point at which it will not abort (terminate without completing).
- Beyond the commit point, the transaction is said to be committed, and its effect is assumed to be permanently recorded in the database.
- Commitment always involves writing a commit entry to the log and writing the log to disk.

TRANSACTION LOGGING

- **Committing Transactions**
- Commitment always involves writing a commit entry to the log and writing the log to disk.
- At the time of a system crash, we search back in the log for all transactions T that have **written a start_transaction(T) entry into the log but have not written a commit(T)** entry yet; these transactions may have to be rolled back to undo their effect on the database during the recovery process.
- Transactions that have **written their commit(T)** entry in the log must also have recorded all their write operations in the log - otherwise **they would not be committed** - so their effect on the database can be redone from the log entries

TRANSACTION LOGGING

- Force-writing
- At the time of a system crash, only the log entries that have been written back to disk are considered in the recovery process, because the contents of main memory may be lost.
- Hence, before a transaction reaches its commit point, any portion of the log that has not been written to the disk yet must now be written to the disk.
- This process is called force-writing of the log file, before committing a transaction.
- A commit does not necessarily involve writing the data items to disk; this depends on the recovery mechanism in use.

TRANSACTION LOGGING

- Checkpoints
- In the event of failure, most recovery managers initiate procedures that involve redoing or undoing operations contained within the log.
- Not all operations need to be redone or undone, as many transactions recorded on the log will have been successfully completed and the changes written permanently to disk.
- The problem for the recovery manager is to determine which operations need to be considered and which can safely be ignored.
- This problem is usually overcome by writing another kind of entry in the log: the checkpoint entry.

TRANSACTION LOGGING

- Checkpoints
- Performing a checkpoint consists of the following operations:
 - Suspending executions of transactions temporarily;
 - Writing (force-writing) all modified database buffers of committed transactions out to disk;
 - Writing a checkpoint record to the log; and
 - Writing (force-writing) all log records in main memory out to disk.
- A checkpoint record usually contains additional information, including a list of transactions active at the time of the checkpoint.
- Many recovery methods (including the deferred and immediate update methods) need this information when a transaction is rolled back, as all transactions active at the time of the checkpoint and any subsequent ones may need to be redone.

TRANSACTION LOGGING

- Undoing
- If a transaction crash does occur, then the recovery manager may undo transactions (that is, reverse the operations of a transaction on the database).
- This involves examining a transaction for the log entry `write_item(T, x, old_value, new_value)` and setting the value of item `x` in the database to `old-value`.
- Undoing a number of `write_item` operations from one or more transactions from the log must proceed in the reverse order from the order in which the operations were written in the log.

TRANSACTION LOGGING

- Redoing
- Redoing transactions is achieved by examining a transactions log entry and for every `write_item(T, x, old_value, new_value)` entry, the value of item `x` in the database is set to `new_value`.
- Redoing a number of transactions from the log must proceed in the same order in which the operations were written in the log.
- Redo operation is required to be **idempotent**; that is, executing it over and over is equivalent to executing it just once.
- The whole recovery process should be idempotent. This is so because, if the system were to fail during the recovery process, the next recovery attempt might redo certain `write_item` operations that had already been redone during the previous recovery process.

TYPES OF FAILURES

1. **Computer failure (system crash)** - hardware, software, or network error occurs in the computer system during transaction execution.
2. **Transaction or system error** - transaction may cause it to fail, such as integer overflow or division by zero. Transaction failure may also occur because of erroneous parameter values or because of a logical programming error
3. **Local errors or exception conditions detected by the transaction** - During transaction execution, certain conditions may occur that necessitate cancellation of the transaction

TYPES OF FAILURES

4. **Concurrency control enforcement** - The concurrency control method may decide to abort a transaction because it violates serializability, or it may abort one or more transactions to resolve a state of deadlock among several transactions. Transactions aborted because of serializability violations or deadlocks are typically restarted automatically at a later time.
5. **Disk failure** - Some disk blocks may lose their data because of a read or write malfunction or because of a disk read/write head crash. This may happen during a read or a write operation of the transaction.

TYPES OF FAILURES

- 6. **Physical problems and catastrophes** - This refers to an endless list of problems that includes power or air-conditioning failure, fire, theft, sabotage overwriting disks or tapes by mistake, and mounting of a wrong tape by the operator.
 - **Non catastrophic Failure** – 1, 2, 3, 4: the system must keep sufficient information to quickly recover from the failure
 - **Catastrophic Failure** – 5 & 6:

RECOVERY CONCEPT

- Recovery from transaction failures usually means that the **database is restored to the most recent consistent state just before the time of failure.**
- To do this, the system, must keep information about the changes that were applied to data items by the various transactions.
- This information is typically kept in the **system log**, stored in onto a storage medium such as magnetic tapes.
- In case of a **catastrophic system failure**, the latest backup copy can be reloaded from the tape to the disk, and the system can be restarted.

RECOVERY CONCEPT

- Typical Strategy for recovery
 - If there is extensive damage to a wide portion of the database due to catastrophic failure, such as a disk crash, the recovery method restores a old copy of the database that was backed up to archival storage (typically tape or other large capacity offline storage media) and reconstructs a more current state by reapplying or redoing the operations of committed transactions from the backed up log, up to the time of failure.

RECOVERY CONCEPT

○ Typical Strategy for recovery

- Database on disk is not physically damaged, and a noncatastrophic failure has occurred, the recovery strategy is to identify any changes that may cause an inconsistency in the database.
- For example, a transaction that has updated some database items on disk but has not been committed needs to have its changes reversed by **undoing** its write operations.
- It may also be necessary to **redo** some operations in order to restore a consistent state of the database.
- For noncatastrophic failure, the recovery protocol does not need a complete archival copy of the database.

RECOVERY TECHNIQUES

- Recovery from noncatastrophic failures – deferred update and immediate update.
- Deferred update techniques:
- Do not physically update the database on disk until after a transaction reaches its commit point; then the updates are recorded in the database.
- Before reaching commit, all transaction updates are recorded in the local transaction workspace or in the main memory buffers that the DBMS maintains (the DBMS main memory cache).

RECOVERY TECHNIQUES

- Deferred update techniques:
- Before commit, the updates are recorded persistently in the log, and then after commit, the updates are written to the database on disk.
- If a transaction fails before reaching its commit point, it will not have changed the database.
- Redo may be necessary for the operations of committed transaction from the log, because the effect may not yet been recorded in the database.
- Referred as differed update is also known as **NO-UNDO/REDO algorithm:**

RECOVERY TECHNIQUES

- Immediate update techniques:
- Database may be updated by some operations of a transaction before the transaction reaches its commit point.
- These operations must also be recorded in the log on disk by **force-writing** before they are applied to the database on disk, making recovery still possible.
- If a transaction fails after recording some changes in the database on disk but before reaching its commit point, the effect of its operations on the database must be undone; that is, the transaction must be rolled back.

RECOVERY TECHNIQUES

- Immediate update techniques:
- The general case of immediate update, both undo and redo may be required during recovery.
- This technique, known as the **UNDO/REDO algorithm**, requires both operations during recovery, and is used most often in practice

RECOVERY TECHNIQUES

- UNDO and REDO operations are required to be **idempotent**—that is, executing an operation multiple times is equivalent to executing it just once.
- The whole recovery process should be idempotent because if the system were to fail during the recovery process, the next recovery attempt might UNDO and REDO certain **write_item** operations that had already been executed during the first recovery process.
- The result of recovery from a system crash during recovery should be the same as the result of recovering when there is no crash during recovery!

RECOVERY CONCEPTS

- **Caching or Buffering of Disk Blocks**
 - Caching of disk pages is traditionally an operating system function, but because of its importance to the efficiency of recovery procedures, it is handled by the DBMS by calling low-level operating systems routines.
 - A collection of in-memory buffers, called the **DBMS cache**, is kept under the control of the DBMS for the purpose of holding these buffers.

RECOVERY CONCEPTS

- Caching or Buffering of Disk Blocks
 - **Directory** for the cache is used to keep track of which database items are in the buffers.
 - This can be a table of **<Disk_page_address, Buffer_location, ... >** entries.
- To **replace** (or **flush**) some of the cache buffers to make space available for the new item.
- DBMS cache has additional informations:
 - **Dirty bit** – to indicate whether or not the buffer has been modified.
 - **Pin-unpin bit** - a page in the cache is pinned, not in cache is unpin.

RECOVERY CONCEPTS

- Caching or Buffering of Disk Blocks
- Strategy to Flushing:
 - **In-place updating**: writes the buffer to the same original disk location, thus overwriting the old value of any changed data item on disk.
 - **Shadowing**: writes an updated buffer at a different disk location, so multiple version of data item can be maintained.
- Old value of the data are referred as **Before Image (BFIM)**
- New value of the data after update as **After Image (AFIM)**

RECOVERY CONCEPTS

- **Write-Ahead Logging**
- Is referred as recovery mechanism must ensure that the BFIM of the data item is recorded in the appropriate log entry and that the log entry is flushed to disk before the BFIM is overwritten with the AFIM in the database on disk.
- Need two type of log entry information:
 - **REDO-type log entry** – includes the new value AFIM of the item written by the operation, used during REDO operation.
 - **UNDO-type log entry** – includes the old value BFIM of the item, which is used during UNDO operation.

RECOVERY CONCEPTS

- Steal/no-steal
- If a cache buffer page updated by a transaction cannot be written to disk before the transaction commits, the recovery method is called a no-steal approach.
- The pin-unpin bit will be used to indicate if a page cannot be written back to disk.
- If the recovery protocol allows writing an updated buffer before the transaction commits, it is called steal.

RECOVERY CONCEPTS

- Steal/no-steal
- Steal is used when the DBMS cache (buffer) manager needs a buffer frame for another transaction and the buffer manager replaces an existing page that had been updated but whose transaction has not committed.
- The no-steal rule means that UNDO will never be needed during recovery, since a committed transaction will not have any of its updates on disk before it commits.

RECOVERY CONCEPTS

- No-force / Force approach
- If all pages updated by a transaction are immediately written to disk before the transaction commits, it is called a **force** approach.
- Otherwise, it is called **no-force**.
- The force rule means that REDO will never be needed during recovery, since any committed transaction will have all its updates on disk before it is committed.

RECOVERY CONCEPTS

- DBMS employ a steal/no-steal strategy.
- NO-UNDO(Deferred update) recovery scheme – no-steal approach.
- Advantage of steal – avoids the need for very large buffer space to store all updated pages in memory.
- Advantage of no-force
 - Updated page of committed transaction remain in buffer when another transaction needs to update, thus avoid multiple I/O cost to write.

RECOVERY CONCEPTS

- Checkpoints in the System Log & Fuzzy Checkpointing
- A checkpoint – list of active transaction record is written into the log periodically at that point when the system writes out to the database on disk.
- Checkpointing: list of transaction ids for active transactions at the time of the checkpoint is included in checkpoint record.
- DMBS decide at what intervals to take a checkpoint.

RECOVERY CONCEPTS

- Checkpoints in the System Log & Fuzzy Checkpointing
- Checkpoint consist of the following actions:
 1. Suspend execution of transactions temporarily.
 2. Force-write all main memory buffers that have been modified to disk.
 3. Write a [checkpoint] record to the log, and force-write the log to disk.
 4. Resume executing transactions.

RECOVERY CONCEPTS

- Checkpoints in the System Log & Fuzzy Checkpointing
- Step 2 makes log to include additional informations like
 - list of active transaction ids
 - Location of first and most recent records for active transaction.
- Step 1 delay the time needed for force-write all modified memory buffer – use a technique called **fuzzy checkpointing**.

RECOVERY CONCEPTS

- Fuzzy Checkpointing
- The system can resume transaction processing after a [begin_checkpoint] record is written to the log without having to wait for step 2 to finish.
- When step 2 is completed, an [end_checkpoint, ...] record is written in the log with the relevant information collected during checkpointing.
- However, until step 2 is completed, the previous checkpoint record should remain valid.

RECOVERY CONCEPTS

- Fuzzy Checkpointing
- To accomplish this, the system maintains a file on disk that contains a pointer to the valid checkpoint, which continues to point to the previous checkpoint record in the log.
- Once step 2 is concluded, that pointer is changed to point to the new checkpoint in the log.

RECOVERY CONCEPTS

- Transaction Rollback and Cascading Rollback
- If a transaction fails for whatever reason after updating the database, but before the transaction commits, it may be necessary to **roll back** the transaction.
- **Cascading Rollback:**
- If a transaction T is rolled back, any transaction S that has, in the interim, read the value of some data item X written by T must also be rolled back.
- Similarly, once S is rolled back, any transaction R that has read the value of some data item Y written by S must also be rolled back; and so on. This is called **Cascading rollback**.

RECOVERY CONCEPTS

- Transaction Rollback and Cascading Rollback
- Cascading rollback - can occur when the recovery protocol ensures recoverable schedules but does not ensure strict or cascadeless schedules.
- Example for cascading rollback with tree transactions T1, T2, T3.
- $A=30, B=15, C=40, D=20$.
- At the point of system failure, T3 has not reached its conclusion and must be rolled back.
- The Writes of T3 that are undone during transaction rollback.

○ Read and Write of three transactions

T_1	T_2	T_3
read_item(A)	read_item(B)	read_item(C)
read_item(D)	write_item(B)	write_item(B)
write_item(D)	read_item(D)	read_item(A)
	write_item(D)	write_item(A)

○ System log at point of crash

	A	B	C	D
	30	15	40	20
	[start_transaction, T_3]			
	[read_item, T_3, C]			
*	[write_item, $T_3, B, 15, 12$]	12		
	[start_transaction, T_2]			
	[read_item, T_2, B]			
**	[write_item, $T_2, B, 12, 18$]	18		
	[start_transaction, T_1]			
	[read_item, T_1, A]			
	[read_item, T_1, D]			
	[write_item, $T_1, D, 20, 25$]			25
	[read_item, T_2, D]			
**	[write_item, $T_2, D, 25, 26$]			26
	[read_item, T_3, A]			

← System crash

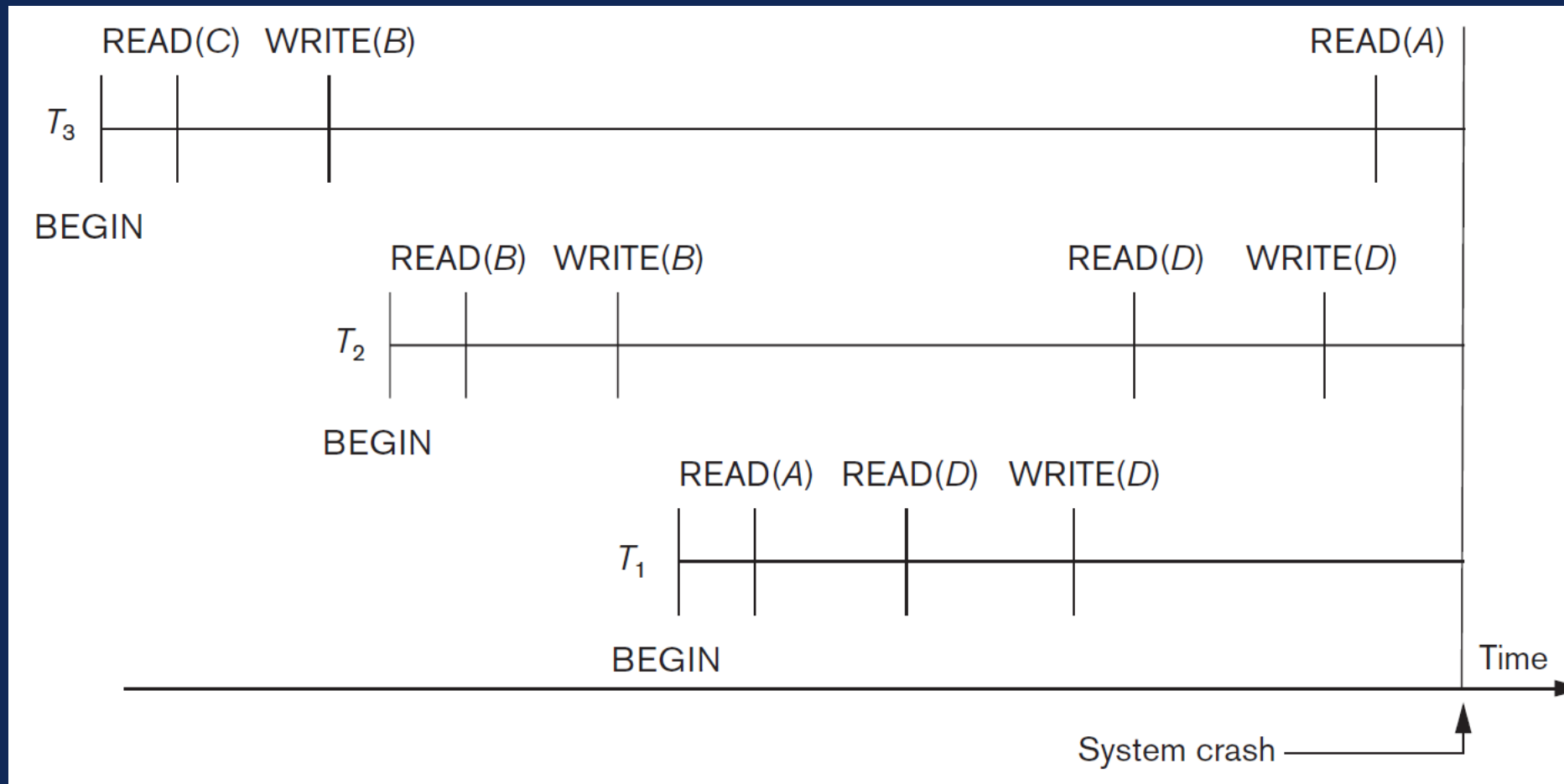
at point of crash. (c)
Operations before the

* T_3 is rolled back because it did not reach its commit point.

** T_2 is rolled back because it reads the value of item B written by T_3 .

RECOVERY CONCEPTS

- Operation before system crash



RECOVERY CONCEPTS

- Time axis figure show how rollback will happen.
- T2 reads the value of item B that was written by transaction T3; this can also be determined by examining the log. Because T3 is rolled back, T2 must now be rolled back, too.
- The WRITE operations of T2, marked by ** in the log, are the ones that are undone.
- Note:
 - Only write_item operations need to be undone during transaction rollback;
 - read_item operations are recorded in the log only to determine whether cascading rollback of additional transactions is necessary.



RECOVERY BASED ON DEFERRED UPDATE

NO-UNDO/REDO RECOVERY BASED ON DEFERRED UPDATE

- The idea behind deferred update is to defer or postpone any actual updates to the database on disk until the transaction completes its execution successfully and reaches its commit point.
- If a transaction fails before reaching its commit point, there is no need to undo any operations.
- Only REDO-type log entries are needed in the log, which include the new value (AFIM) of the item written by a write operation.
- UNDO-type log entries are not needed since no undoing of operations will be required during recovery.

NO-UNDO/REDO RECOVERY BASED ON DEFERRED UPDATE

55

- The steps involved in the deferred update protocol are as follows:
 1. When a transaction starts, write an entry `start_transaction(T)` to the log.
 2. When any operation is performed that will change values in the database, write a log entry `write_item(T, x, old_value, new_value)`.
 3. When a transaction is about to commit, write a log record of the form `commit(T)`; write all log records to disk.
 4. Commit the transaction, using the log to write the updates to the database; the writing of data to disk need not occur immediately.
 5. If the transaction aborts, ignore the log records and do not write the changes to disk.

NO-UNDO/REDO RECOVERY BASED ON DEFERRED UPDATE

- The database is never updated until after the transaction commits, and there is no need to UNDO any operations.
- Hence this technique is known as the **NO-UNDO/REDO algorithm**.
- The REDO is needed in case the system fails after the transaction commits but before all its changes are recorded in the database.

NO-UNDO/REDO RECOVERY BASED ON DEFERRED UPDATE

- The protocol and how different entries are affected can be best summarised as shown:

Log entry	Log written to disk	Changes written to database buffer	Changes written on disk
start_transaction(T)	No	N/A	N/A
read_item(T, x)	No	N/A	N/A
write_item(T, x)	No	No	No
commit(T)	Yes	Yes	*Yes
checkpoint	Yes	Undefined	Yes(of committed Ts)

*Yes: writing back to disk may occur not immediately.

DEFERRED UPDATE IN A SINGLE-USER ENVIRONMENT

- Recovery based on deferred update in single-user systems, where no concurrency control is needed.
- Use two lists to maintain the transactions:
 - committed transactions list, which contains all the committed transactions since the last checkpoint,
 - active transactions list (at most one transaction falls in this category, because the system is a single-user one).
- Apply the REDO operation to all the `write_item` operations of the committed transactions from the log in the order in which they were written to the log.
- Restart the active transactions.

DEFERRED UPDATE IN A SINGLE-USER ENVIRONMENT

59

- REDO procedure is defined as follows:
- Redoing a `write_item` operation consists of
 - examining its log entry `write_item(T, x, old_value, new_value)`
 - setting the value of item `x` in the database to `new_value`.
- The REDO operation is required to be idempotent.
- Note:
 - List of active transactions will have no effect on the database and is ignored completely by the recovery process.
 - It is implicitly rolled back, because none of its operations were reflected in the database.
- However, the transaction must now be restarted, either automatically or manually.

DEFERRED UPDATE IN A SINGLE-USER ENVIRONMENT

- Main benefit is that any transaction operation need never be undone, as a transaction does not record its changes in the database until it reaches its commit point.
- The protocol is summarised in the diagram below:

Action	Entry in log	
	start_transaction(T)	commit(T)
Re-submit	Yes	No
Redo	Yes	Yes

DEFERRED UPDATE IN A SINGLE-USER ENVIRONMENT

- Figure below shows an example of recovery in a single-user environment, where the first failure occurs during execution of transaction T2.

start_transaction(T1)	
write_item(T1, D, 20)	
commit(T1)	
start_transaction(T2)	
write_item(T2, B, 10)	
write_item(T2, D 25)	← System crash

T ₁	T ₂
read_item(A)	read_item(B)
read_item(D)	write_item(B)
write_item(D)	read_item(D)
	write_item(D)

DEFERRED UPDATE IN A SINGLE-USER ENVIRONMENT

- The recovery process will redo the `write_item(T1, D, 20)` entry in the log by resetting the value of item D to 20 (its new value).
- The `write(T2, D 25)` entries in the log are ignored by the recovery process because T2 is not committed.
- If a second failure occurs during recovery from the first failure, the same recovery process is repeated from start to finish, with identical results.

DEFERRED UPDATE IN A MULTI-USER ENVIRONMENT

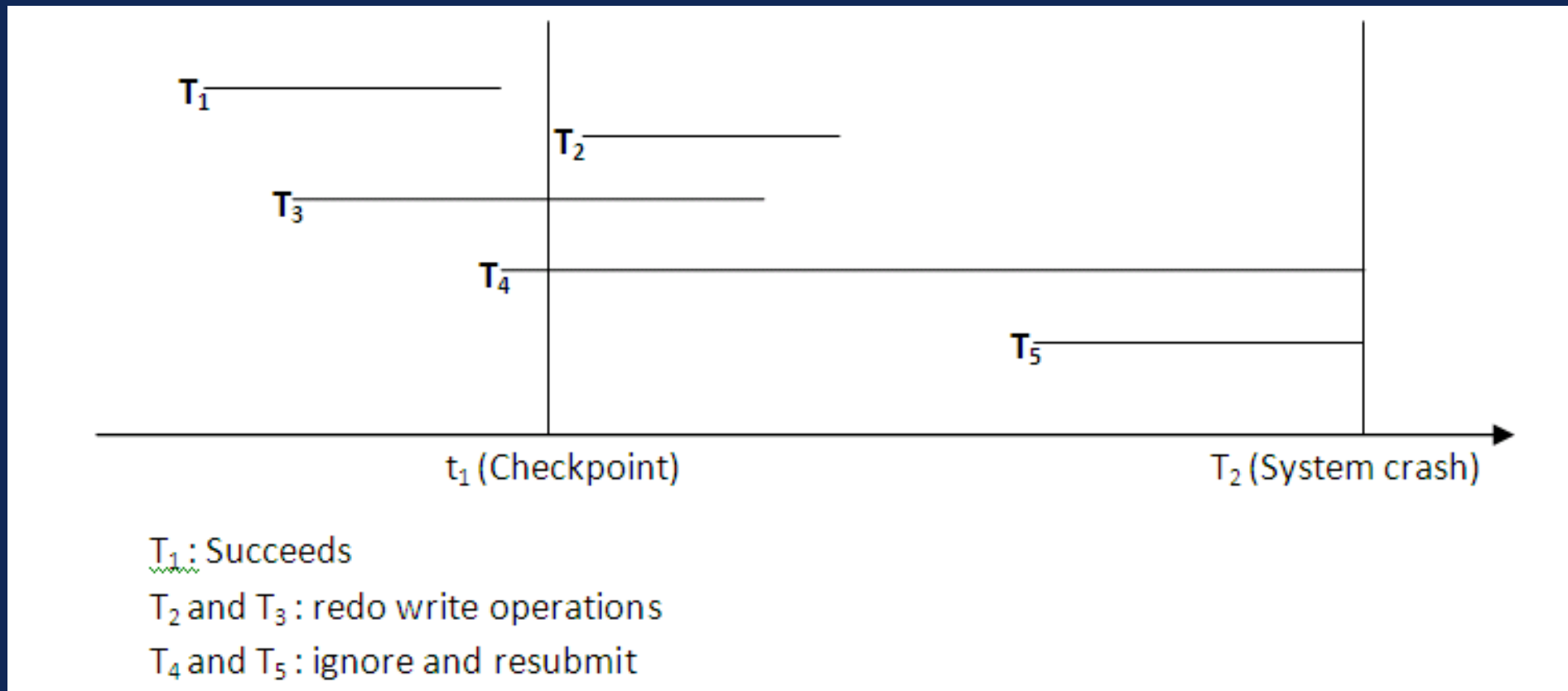
- For a multi-user system with concurrency control, the recovery process may be more complex, depending on the protocols used for concurrency control.
- Concurrency control uses two-phase locking (basic 2PL) and prevents deadlock by pre-assigning all locks to items needed by a transaction before the transaction starts execution.
- To combine the deferred update methods for recovery with this concurrency control technique, we can keep all the locks on items in effect until the transaction reaches its commit point.

DEFERRED UPDATE IN A MULTI-USER ENVIRONMENT

- After that, the locks can be released.
- This ensures strict and serializable schedules.
- Assuming that checkpoint entries are included in the log, a possible recovery algorithm for this case is given below.
 - Use two lists of transactions maintained by the system: the **committed transactions list** which contains all committed transactions since the last checkpoint, and the **active transactions list**.
 - REDO all the write operations of the committed transactions from the log, in the order in which they were written into the log.
 - The transactions in the **active list** that are active and did not commit **are effectively cancelled and must be resubmitted**.

DEFERRED UPDATE IN A MULTI-USER ENVIRONMENT

- The REDO procedure is the same as defined earlier in the deferred update in the single-user environment.
- Consider the schedule of executing transactions:



DEFERRED UPDATE IN A MULTI-USER ENVIRONMENT

- When the checkpoint was taken at time t_1 , transaction T_1 had committed, whereas transaction T_3 and T_4 had not.
- Before the system crash at time t_2 , T_3 and T_2 were committed but not T_4 and T_5 .
- According to the deferred update method, there is no need to redo the write operations of transaction T_1 or any transactions committed before the last checkpoint time t_1 .

DEFERRED UPDATE IN A MULTI-USER ENVIRONMENT

- Write operations of T2 and T3 must be redone, however, because both transactions reached their commit points after the last checkpoint.
- Recall that the log is force-written before committing a transaction.
- Transaction T4 and T5 are ignored: they are effectively cancelled and rolled back because none of their write operations were recorded in the database under the deferred update protocol.

EXERCISE

- Given the operations of the four concurrent transactions in Figure. 1 below and the system log at the point of system crash in Figure .2, discuss how each transaction recovers from the failure using deferred update technique.
- Figure 1:

T₁	T₂	T₃	T₄
read_item(A)	read_item(B)	read_item(A)	read_item(B)
read_item(D)	write_item(B)	write_item(A)	write_item(B)
write_item(D)	read_item(D)	read_item(C)	read_item(A)
	write_item(D)	write_item(C)	write_item(A)

EXERCISE

○ Figure 2:

start_transaction(T_1)	
write_item(T_1 , D, 20)	
commit(T_1)	
checkpoint	
start_transaction(T_4)	
write_item(T_4 , B, 15)	
commit(T_4)	
start_transaction(T_2)	
write_item(T_2 , B, 12)	
start_transaction(T_3)	
write_item(T_3 , A, 30)	
write_item(T_2 , D, 25)	← System crash

EXERCISE

- Solution:
- T1 has committed.
- T4 has committed after checkpoint.
- T3 and T2 are not committed.
- According to Deferred updated:
- No need to redo T1.
- Write of T4 must be redone, committed after checkpoint.
- T2 and T3 are cancelled and rolledback.

start_transaction(T ₁)	
write_item(T ₁ , D, 20)	
commit(T ₁)	
checkpoint	
start_transaction(T ₄)	
write_item(T ₄ , B, 15)	
commit(T ₄)	
start_transaction(T ₂)	
write_item(T ₂ , B, 12)	
start_transaction(T ₃)	
write_item(T ₃ , A, 30)	
write_item(T ₂ , D, 25)	← System crash



RECOVERY TECHNIQUES BASED ON IMMEDIATE UPDATE

RECOVERY TECHNIQUES BASED ON IMMEDIATE UPDATE

- In the immediate update techniques, the database may be updated by the operations of a transaction immediately, before the transaction reaches its commit point.
- However, these operations are typically recorded in the log on disk by force-writing before they are applied to the database, so that recovery is possible.

RECOVERY TECHNIQUES BASED ON IMMEDIATE UPDATE

- When immediate update is allowed, provisions must be made for undoing the effect of update operations on the database, because a transaction can fail after it has applied some updates to the database itself.
- Hence recovery schemes based on immediate update must include the capability to roll back a transaction by undoing the effect of its write operations.

RECOVERY TECHNIQUES BASED ON IMMEDIATE UPDATE

1. When a transaction starts, write an entry `start_transaction(T)` to the log;
2. When any operation is performed that will change values in the database, write a log entry `write_item(T, x, old_value, new_value)`;
3. Write the log to disk;
4. Once the log record is written, write the update to the database buffers;
5. When convenient, write the database buffers to the disk;
6. When a transaction is about to commit, write a log record of the form `commit(T)`;
7. Write the log to disk.

RECOVERY TECHNIQUES BASED ON IMMEDIATE UPDATE

- The protocol and how different entries are affected can be best summarised below:

Log entry	Log written to disk	Changes written to database buffer	Changes written on disk
start_transaction(T)	No	N/A	N/A
read_item(T, x)	No	N/A	N/A
write_item(T, x)	Yes	Yes	*Yes
commit(T)	Yes	Undefined	Undefined
Checkpoint	Yes	Undefined	Yes(of committed Ts)

*Yes: writing back to disk may not occur immediately

RECOVERY TECHNIQUES BASED ON IMMEDIATE UPDATE

- Two main categories of immediate update algorithms.
 - If the recovery technique ensures that **all updates of a transaction are recorded in the database on disk** before the transaction commits, there is **no need to redo any operations** of committed transactions. Such an algorithm is called **UNDO/NO-REDO**.
 - On the other hand, if the **transaction is allowed to commit before all its changes are written to the database**, we have the **UNDO/REDO** method, the most general recovery algorithm.
 - This is also the most complex technique.
- Recovery activities are:

Action	Entry in log	
	start_transaction(T)	commit(T)
Undo and resubmit T	Yes	No
Redo	Yes	Yes

IMMEDIATE UPDATE IN A MULTI-USER ENVIRONMENT

77

- When concurrency execution is permitted, the recovery process again depends on the protocols used for concurrency control.
- The procedure below outlines a recovery technique for concurrent transactions with immediate update.
- Assume that the log includes checkpoints and that the concurrency control protocol produces strict schedules, for example, the strict 2PL protocol does.
- Recall that a strict schedule does not allow a transaction to read or write an item unless the transaction that last wrote the item has committed.
- However, deadlocks can occur in strict 2PL, thus requiring UNDO of transactions.

IMMEDIATE UPDATE IN A MULTI-USER ENVIRONMENT

- When concurrency execution is permitted, the recovery process again depends on the protocols used for concurrency control.
- The procedure below outlines a recovery technique for concurrent transactions with immediate update.
- Assume that the log includes checkpoints and that the concurrency control protocol produces strict schedules, for example, the strict 2PL protocol does.
- Recall that a strict schedule does not allow a transaction to read or write an item unless the transaction that last wrote the item has committed.



SHADOW PAGING

SHADOW PAGING

- In the shadow page scheme, the database is not directly modified but a copy, stored on permanent storage (e.g. disk), is made of the portion of the database to be modified and all modifications are made to this copy.
- Meanwhile, the old version of the database remains intact.
- Once the transaction commits, the modified copy replaces the original in an atomic manner, i.e. the replacement is carried out in its entirety or not at all.
- If a system crashes at this point, the old version is still available for recovery.

SHADOW PAGING

- Page management
- Paging scheme as used in the operating system for virtual memory management is appropriate.
- The memory that is accessed by a process (a program in execution is a process) is called virtual memory.
- Virtual memory is divided into pages that are all of a certain size (commonly 4096 bytes or 4K).
- The virtual or logical pages are mapped onto physical memory blocks (i.e. disk physical records) of the same size as the pages.

SHADOW PAGING

- Page management
- The mapping is achieved by consulting a page table (or directory).
- The page table lists each logical page identifier and the address of the physical blocks that actually stores the logical page.
- The advantage of this scheme is that the consecutive logical pages need not be mapped onto consecutive physical blocks.

SHADOW PAGING

- Shadow paging scheme considers the database to be made up of a number of fixed-size disk pages (or disk blocks) say, n for recovery purposes.
- A page table (or directory) with n entries is constructed, where the i th page table entry points to the i th database page on disk.
- The page table is kept in main memory if it is not too large, and all references - reads or writes - to database pages on disk go through the page table.

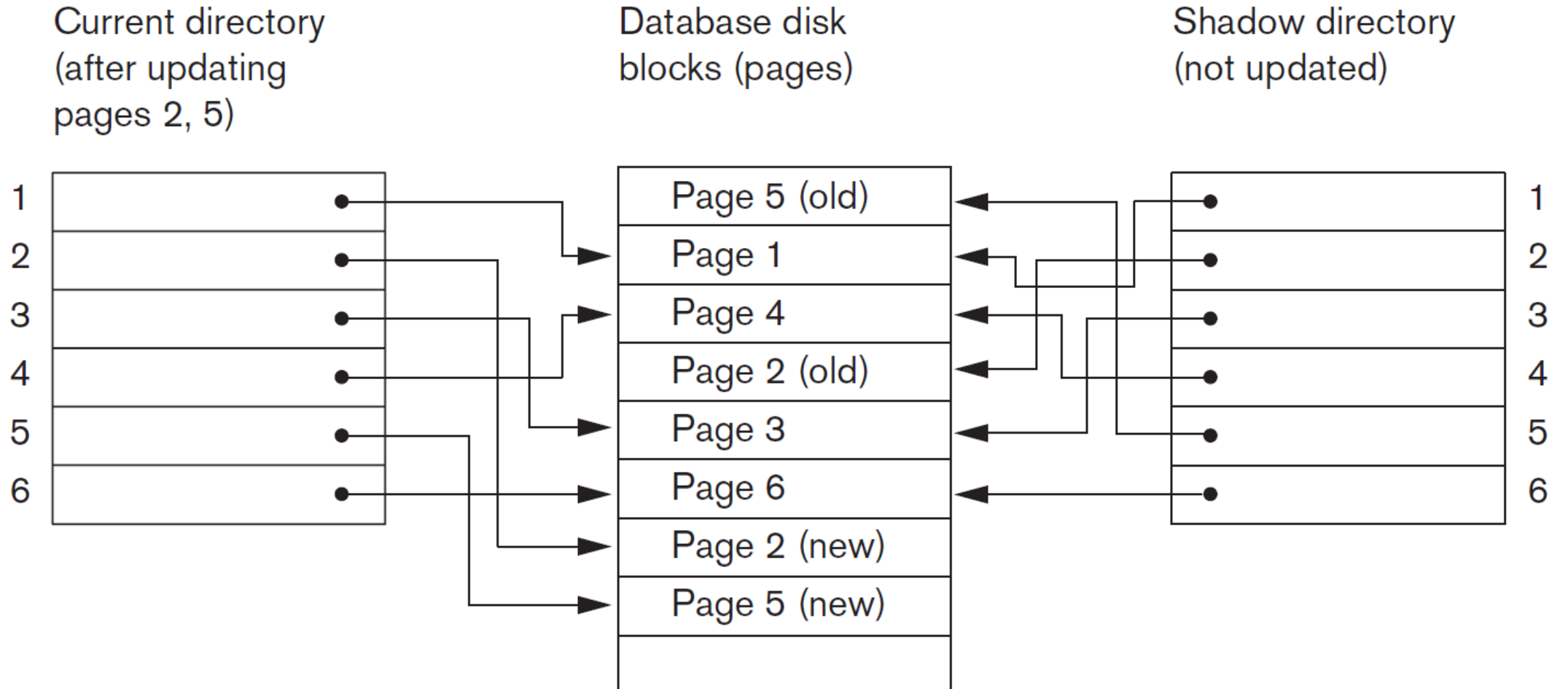
SHADOW PAGING

- When a transaction begins executing, the **current directory**—whose entries point to the most recent or current database pages on disk—is copied into a **shadow directory**.
- The shadow directory is then saved on disk while the current directory is used by the transaction.
- During transaction execution, the shadow directory is never modified.
- When a `write_item` operation is performed, a new copy of the modified database page is created, but the old copy of that page is not overwritten.

SHADOW PAGING

- The current directory entry is modified to point to the new disk block, whereas the shadow directory is not modified and continues to point to the old unmodified disk block.
- For pages updated by the transaction, two versions are kept.
- The old version is referenced by the shadow directory and the new version by the current directory.

SHADOW PAGING



SHADOW PAGING SCHEME IN A SINGLE-USER ENVIRONMENT

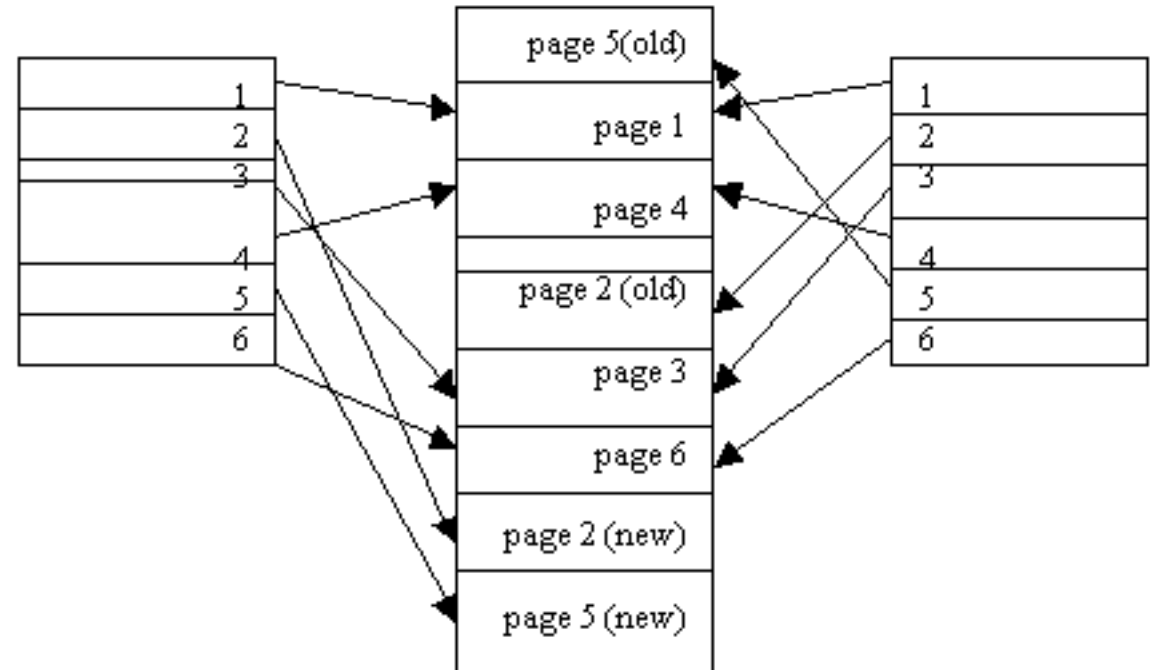
- In the shadow page scheme, two page tables are used.
- The original page table (shadow page table) and the current page table.
- Initially, both page tables point to the same blocks of physical storage.
- The current page table is the only route through which a transaction can gain access to the data stored on disk.
- That is, a transaction always uses the current page table to retrieve the appropriate database blocks.

SHADOW PAGING SCHEME IN A SINGLE-USER ENVIRONMENT

- During transaction execution, the shadow page table is never modified.
- When a write operation is performed, a new copy of the modified database page is created, but the old copy of that page is not overwritten.
- Instead, the new page is written elsewhere “ on some previously unused disk block.
- The current page table entry is modified to point to the new disk block, whereas the shadow page table is not modified and continues to point to the old unmodified disk block.

SHADOW PAGING SCHEME IN A SINGLE-USER ENVIRONMENT

- The diagram above illustrates the concepts of a shadow page table and a current page table.
- For pages updated by the transaction, two versions are kept.
- The old version is referenced by the shadow page table and the new version by the current page table.



SHADOW PAGING SCHEME IN A SINGLE-USER ENVIRONMENT

- To recover from a failure during transaction execution, it is sufficient to free the modified database pages and to discard the current page table.
- The state of the database before transaction execution is available through the shadow page table, and that state is recovered by reinstating the shadow page table so that it becomes the current page table once more.

SHADOW PAGING SCHEME IN A SINGLE-USER ENVIRONMENT

91

- The database thus is returned to its state prior to the transaction that was executing when the crash occurred, and any modified pages are discarded.
- Committing a transaction corresponds to discarding the previous shadow page table and freeing old page tables that it references.
- Since recovery involves neither undoing nor redoing data items, this technique is called the NO-UNDO/NO-REDO recovery technique.

SHADOW PAGING SCHEME IN A SINGLE-USER ENVIRONMENT

- Advantage:
- Shadow paging is that it makes undoing the effect of the executing transaction very simple.
- There is no need to undo or redo any transaction operations.
- In a multi-user environment with concurrent transactions, logs and checkpoints must be incorporated into the shadow paging technique.
- To handle garbage collection when a transaction commits.

SHADOW PAGING SCHEME IN A SINGLE-USER ENVIRONMENT

- Disadvantage:
- Shadow paging is that the updated database pages change location on disk.
- Makes it difficult to keep related database pages close together on disk without complex storage management strategies.
- If the page table (directory) is large, the overhead of writing shadow page tables to disk as transactions commit is significant.



THANK YOU

BCSE304L – Database Systems
Module 5